

```
package jp.co.active.kenshu.diary;

import java.time.LocalDate;
import java.util.HashMap;
import java.util.Map;

/**
 * 日記入力のバリデーションを行うヘルパークラス
 */
public class DiaryValidationHelper {

    /**
     * 年月日をチェックして LocalDate 型に変換する
     *
     * ※例外処理はここではしない
     * ※発生した例外は ConfirmServlet で catch する
     *
     * @param year 年
     * @param month 月
     * @param day 日
     * @return LocalDate 型の日付
     */
    public static LocalDate createDate(String year, String month, String day) {

        // 年月日が null または空文字の場合
        if (isBlank(year) || isBlank(month) || isBlank(day)) {
            throw new IllegalArgumentException("年月日が正しくありません。");
        }

        // 年月日が数字以外の場合
        if (!isNumber(year) || !isNumber(month) || !isNumber(day)) {
            throw new NumberFormatException("年月日は数字で指定してください。");
        }

        int yearNumber = Integer.parseInt(year);
        int monthNumber = Integer.parseInt(month);
        int dayNumber = Integer.parseInt(day);

        // 存在しない年月日の場合は、ここで DateTimeException が発生する
        return LocalDate.of(yearNumber, monthNumber, dayNumber);
    }
}
```

```
}
```

```
/**
```

```
 * 日記内容の入力チェックを行う
```

```
 *
```

```
 * @param title タイトル
```

```
 * @param main 本文
```

```
 * @param feel 今日の気分
```

```
 * @param weather 今日の天気
```

```
 * @return エラーメッセージ
```

```
 */
```

```
public static Map<String, String> validateDiary(  
    String title,  
    String main,  
    String feel,  
    String[] weather) {
```

```
    Map<String, String> errors = new HashMap<>();
```

```
    // タイトル未入力チェック
```

```
    if (isBlank(title)) {  
        errors.put("title", "※必須入力です。");  
    }
```

```
    // 本文未入力チェック
```

```
    if (isBlank(main)) {  
        errors.put("main", "※必須入力です。");  
    }
```

```
    // 気分未選択チェック
```

```
    if (isBlank(feel)) {  
        errors.put("feel", "※必須選択です。");  
    }
```

```
    // 気分に想定外の値が渡された場合
```

```
    } else if (!isValidFeel(feel)) {  
        errors.put("feel", "不正な値が送信されました。");  
    }
```

```
    // 天気未選択チェック
```

```
    if (weather == null || weather.length == 0) {  
        errors.put("weather", "※必須選択です。");  
    }
```

```
// 天気に想定外の値が渡された場合
} else if (!isValidWeather(weather)) {
    errors.put("weather", "不正な値が送信されました。");
}

return errors;
}
```

```
/**
 * 空文字またはnullか判定する
 *
 * @param value 判定する文字列
 * @return nullまたは空文字なら true
 */
private static boolean isBlank(String value) {
    return value == null || value.trim().isEmpty();
}
```

```
/**
 * 数字だけか判定する
 *
 * @param value 判定する文字列
 * @return 数字だけなら true
 */
private static boolean isNumber(String value) {
    return value.matches("[0-9]+");
}
```

```
/**
 * 気分の値が想定内か判定する
 *
 * @param feel 今日の気分
 * @return 想定内なら true
 */
private static boolean isValidFeel(String feel) {
    return "good".equals(feel)
        || "soso".equals(feel)
        || "bad".equals(feel);
}
```

```

/**
 * 天気の値が想定内か判定する
 *
 * @param weather 今日の天気
 * @return 想定内なら true
 */
private static boolean isValidWeather(String[] weather) {
    for (String value : weather) {
        if (!"晴".equals(value)
            && !"曇".equals(value)
            && !"雨".equals(value)
            && !"雪".equals(value)) {
            return false;
        }
    }

    return true;
}

```

```

/**
 * 配列の中に指定した値が含まれているか判定する
 *
 * @param values 配列
 * @param target 探す値
 * @return 含まれていれば true
 */
public static boolean contains(String[] values, String target) {
    if (values == null) {
        return false;
    }

    for (String value : values) {
        if (target.equals(value)) {
            return true;
        }
    }

    return false;
}
}

```